Complexity Analysis Report

Campus resource reservation system

Project Overview:

This campus resource reservation system was created for managing campus resources and student reservation. This program used different data structures such as vector to store resources, linked list for managing reservation, queue for waiting student, and stacks for cancellation. Also, linear search merge sort for searching and sorting resources.

This report explains the time and space complexity for main operation of the program

Time Complexity Analysis:

Operation	Time Complexity	Explanation
Load Resource	O(n)	It reads resources from text file and store them in vector.
Display resources	O(n)	Goes through each resources and display.
Check Availability	O(n)	Check availability status of each resource
Linear search	O(n)	Search resources one by one until it found correct ID
Merge Sort	O(n log n)	Divide resource into small part and merge them in sorted order
Reservation Insertion	O(1)	A linked-list node can be inserted directly in constant time. If the program first scans all active reservations to prevent duplicate reservation IDs, the validation dominates and makes the complete operation linear
Reservation cancellation	O(n)	The linked list may need to be traversed until the requested reservation is found. Once found, changing links is O(1), but the search makes the total O(n).
Search reservation	O(r)	Search the active reservation lined list by reservation ID

Add to Waiting list	O(1)	Add student tot the end of the queue
Undo cancellation	O(n+r)	The most recent cancellation is at the top of the stack, so top/pop is constant time. Reinserting it into the active list can also be O(n+r) when inserted directly; any full-list validation would add O(n)

n = number of campus resources, r = number of active reservation

Searching and Sorting:

Linear Search – O(n):

In this program we used linear search to find campus resources by their ID. The program goes through each resource until it finds the correct ID. In the worst case, it may check every resource, so the time complexity is O(n).

Merge Sort-O(nlog n):

In this system we used merge sort to arrange resources in ascending order. The algorithm divides the resource list into smaller parts sort them and merges them back together. So, the time complexity is O(nlog n).

Reservation Insertion:

Inserting a new node into a linked list can be O(1) when the insertion point is already known, such as inserting at the head or at a maintained tail. However, the project also requires duplicate reservation IDs to be prevented. If duplicate checking is performed by scanning the active-reservation linked list, that check is O(n), so the full create-reservation operation becomes O(n).

Reservation Removal / Cancellation:

To cancel a reservation by ID, the program may need to examine active reservations one by one until it finds the matching node. In the worst case the target is the last node or is not present, so the search takes O(n). Removing the located node itself is O(1), making the overall operation O(n).

Waiting-List Processing:

The waiting list follows FIFO order. Enqueue places a student at the rear, and dequeue removes the student at the front. With normal queue front/rear management, each of these operations is O(1). If the program displays every student in the waiting list, that display operation is O(w).

Undo Cancellation:

Cancellation history follows LIFO order, so the most recently cancelled reservation is always at the top of the stack. Accessing and popping that item is O(1). Restoring it to the active-reservation list can remain O(1) for direct insertion; if restoration performs duplicate validation by scanning the list, the complete restore can become O(n).

Conclusion:

The queue and stack provide efficient constant-time operations for waiting-list processing and cancellation undo. Linked-list insertion can also be constant time, but searching or validating across active reservations requires linear time. Therefore, the main O(n) work in Milestone 1 comes from operations that must scan the active-reservation linked list.